

DevOps Journey - Day 2: Shell Scripting

Suraj Rane

Day 2: Functions, Argument Parsing (getopts), Reading Files Line-by-Line, and Cron

Part 1: Functions - Going Deeper

1.1 Return Values vs Exit Status

A shell function doesn't "return" data the way a Python/JS function does - return in bash only returns an **exit status** (0-255), not a value.

```
is_even() {  
    if (( $1 % 2 == 0 )); then  
        return 0    # success/true  
    else  
        return 1    # failure/false  
    fi  
}  
  
if is_even 4; then  
    echo "Even"  
fi
```

To actually get *data* back out of a function, echo it and capture with command substitution:

```
get_hostname() {  
    echo "server-$1"  
}  
  
name=$(get_hostname "prod")  
echo "$name"    # server-prod
```

1.2 Local Variables

By default, variables inside a function are **global** - this bites people constantly. Always use local:

```
count=10  
  
update_count() {  
    local count=5    # this "count" is scoped to the function only  
    echo "Inside: $count"
```

```

}

update_count
echo "Outside: $count" # still 10, untouched

```

1.3 Passing Arguments to Functions

Functions get their own \$1, \$2, \$@, \$# - separate from the script's:

```

backup_file() {
    local src="$1"
    local dest="$2"
    cp "$src" "$dest"
    echo "Backed up $src to $dest"
}

backup_file "app.log" "app.log.bak"

```

1.4 A Practical Multi-Function Script Structure

This is the pattern real production scripts follow:

```

#!/bin/bash
set -euo pipefail

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1"
}

check_disk_space() {
    local threshold=1
    local usage
    usage=$(df / | awk 'NR==2 {print $5}' | tr -d '%')
    if (( usage > threshold )); then
        log "WARNING: Disk usage at ${usage}%"
        return 1
    fi
    log "Disk usage OK at ${usage}%"
    return 0
}

main() {
    log "Starting health check"
    check_disk_space 80
    log "Health check complete"
}

main

```

Notice: a log() helper, a task function, and a main() that orchestrates - this pattern scales well as scripts grow.

Part 2: Argument Parsing with getopt

So far you've used \$1, \$2 positionally. That's fine for 1-2 args, but real tools take flags: ./script.sh -f app.log -t 80 -v. That's what getopt is for.

2.1 Basic Pattern

```
#!/bin/bash
set -euo pipefail

verbose=false
threshold=80
file=""

while getopts "f:t:v" opt; do
    case $opt in
        f) file="$OPTARG" ;;
        t) threshold="$OPTARG" ;;
        v) verbose=true ;;
        \?) echo "Invalid option: -$OPTARG" >&2; exit 1 ;;
        :) echo "Option -$OPTARG requires an argument" >&2; exit 1
    ;;
esac
done

echo "File: $file"
echo "Threshold: $threshold"
echo "Verbose: $verbose"
```

Run it as:

```
./script.sh -f app.log -t 90 -v
```

2.2 How the getopts String Works

"f:t:v" breaks down as: - f: -> -f takes an argument (the colon means "expects a value") - t: -> -t takes an argument - v -> -v is just a flag, no value (no colon)

2.3 Key Variables Inside the Loop

- \$opt - the flag that was matched
- \$OPTARG - the value passed with that flag (only relevant when the flag expects one)
- \$OPTIND - index of the next argument to process (useful for remaining positional args after flags)

2.4 Combining Flags with Positional Args

```
while getopts "vf:" opt; do
    case $opt in
        v) verbose=true ;;
        f) file="$OPTARG" ;;
    esac
done
shift $((OPTIND - 1)) # removes processed flags, leaves positional
args in $1, $2...

echo "Remaining args: @$@"
```

This shift \$((OPTIND - 1)) line is the part people always forget - without it, \$1 still points at the first flag, not the leftover positional arguments.

Part 3: Reading Files Line-by-Line

You've used `grep/awk` to process files in bulk. Sometimes you need to process each line individually with custom logic - that's where a `while` read loop comes in.

3.1 The Standard Safe Pattern

```
while IFS= read -r line; do
    echo "Processing: $line"
done < "app.log"
```

Why each piece matters: `- IFS=` - prevents leading/trailing whitespace from being stripped - `- r` - prevents backslashes in the line from being interpreted as escape characters - `< "app.log"` - redirects the file as input to the loop (not piped, so variables set inside the loop persist after)

3.2 The Pipe Version Gotcha

```
# BAD - variables set inside won't persist outside
count=0
cat app.log | while read -r line; do
    count=$((count+1))
done
echo "$count"    # prints 0! because the pipe runs the loop in a
subshell

# GOOD - use input redirection instead
count=0
while read -r line; do
    count=$((count+1))
done < app.log
echo "$count"    # correct count
```

This subshell issue is one of the most common bash bugs - worth remembering permanently.

3.3 Practical Example - Line-by-Line with Logic

```
while IFS= read -r line; do
    if [[ "$line" == *ERROR* ]]; then
        echo "ALERT: $line"
    fi
done < app.log
```

3.4 Reading with a Line Number

```
line_num=0
while IFS= read -r line; do
    line_num=$((line_num+1))
    echo "$line_num: $line"
done < app.log
```

Part 4: Cron - Scheduling Your Scripts

All the scripts you're writing are only useful in production if they run automatically. Cron is how.

4.1 Cron Syntax

```
* * * * * /path/to/script.sh
| | | | |
| | | | | +-- day of week (0-6, Sunday=0)
| | | +---- month (1-12)
| | +----- day of month (1-31)
| +----- hour (0-23)
+----- minute (0-59)
```

4.2 Common Examples

```
# Every day at 2:30 AM
30 2 * * * /home/user/scripts/backup.sh

# Every 15 minutes
*/15 * * * * /home/user/scripts/health_check.sh

# Every weekday (Mon-Fri) at 9 AM
0 9 * * * 1-5 /home/user/scripts/report.sh

# First day of every month at midnight
0 0 1 * * /home/user/scripts/monthly_cleanup.sh
```

4.3 Editing Your Crontab

```
crontab -e    # opens your crontab in an editor
crontab -l    # lists current cron jobs
crontab -r    # removes all cron jobs (careful!)
```

4.4 A Critical Production Habit - Logging Cron Output

Cron jobs run in a minimal environment with no terminal, so if something fails silently, you'll never know unless you log it:

```
30 2 * * * /home/user/scripts/backup.sh >> /var/log/backup.log 2>&1
```

- >> appends stdout to the log file
- 2>&1 redirects stderr into the same place as stdout, so errors get captured too

4.5 Common Cron Gotcha

Cron doesn't load your normal shell environment (.bashrc, PATH, etc.), so scripts that work fine manually sometimes fail under cron. Always use **full absolute paths** for commands and files inside cron-run scripts, and consider sourcing your environment explicitly at the top of the script if needed.

Part 5: Hands-On Assignment

Extend yesterday's `error_summary.sh` into a more robust tool called `log_monitor.sh`:

1. Restructure it using functions (`log()`, `check_errors()`, `main()`)
2. Add `getopts` support for:
 - `-f <file>` - log file to check (required)
 - `-t <threshold>` - alert if error count exceeds this number (default 5)
 - `-v` - verbose mode, print each matching line as it's found
3. Use a `while read -r line` loop (with input redirection, not a pipe) to scan the file line-by-line and count ERROR lines
4. If error count exceeds the threshold, log a warning and exit 1; otherwise exit 0
5. Once it works, add a crontab entry (you don't have to actually deploy it - just write out the correct cron line) that would run it every 15 minutes and log output to `/var/log/log_monitor.log`

End of Day 2 - DevOps Journey (55-Day Roadmap)